

FASTER

Platform Documentation

Version 1.0

Contents

1. Platform Overview	3
2. Workspace Navigation	3
3. Quickstart	3
4. Authentication and Access	3
5. config.yaml Structure	3
6. Methods and Comparability	5
7. Parameter Reference	6
8. Datasets and Upload Rules	9
9. Default Network Architectures	10
10. Custom Model Usage	12
11. Scenario Management	13
12. Live Observability	14
13. History and Comparison	14
14. System Status	15
15. Reproducibility	15
16. Troubleshooting	15

Platform Overview

FASTER is a federated learning platform for running, monitoring, and comparing experiments from a single interface. You configure a run, launch it, watch it train, and analyze the results.

Each run begins with a short server warm-up, then trains in rounds. After every round the global model is aggregated and evaluated on the validation set, and on the test set when enabled.

Workspace Navigation

Once signed in, the workspace has five areas:

- **Experiment Setup:** configure and launch a run. Simple mode for the essentials, Advanced mode for full control.
- **Live Monitor:** metrics, logs, and progress while a run is training.
- **Datasets:** upload, search, and delete your custom datasets.
- **History:** review past runs, compare them, and export them.
- **System Status:** CPU, RAM, and GPU usage, and the run queue.

Quickstart

- **Sign in:** authenticate with your assigned account.
- **Open Setup:** choose method, dataset, and training parameters.
- **Load scenario (optional):** apply a saved scenario or upload a YAML file.
- **Launch:** start the run from Setup.
- **Monitor:** track progress and logs in Live Monitor.
- **Compare:** in History, select at least 2 runs to render comparison charts.

Authentication and Access

Accounts are created by administrators; there is no public sign-up. Once signed in, you can change your own password from the workspace.

You see and manage only your own runs, exports, and datasets.

config.yaml Structure

Every run is defined by a YAML configuration, validated before launch. You normally fill it through the Setup form, but you can also import or export it as a file.

Save Defaults remembers the current form as your personal starting point for future sessions. **Save Scenario** stores a named preset you load on demand. The two are independent: defaults prefill automatically, scenarios are loaded explicitly.

The block below shows the full configuration, including advanced and optional fields.

Expand full config.yaml structure

```
# Core identity
# Single method (this is what each launched run actually carries):
method: fed_avg
# Multiple methods (optional): launches one independent run per method,
# sharing every other field below. The UI expands 'methods' into one run per
# entry, each carrying a single 'method'. When 'methods' has one entry it
# equals 'method'.
methods: [fed_avg, fed_prox, fed_gp]
# One run name per method, in the same order (optional). Leave empty for auto:
run_names: avg_baseline, prox_baseline, gp_baseline
dataset_name: pathmnist
evaluation_split_mode: train_val_test

# Scale and scheduling
num_clients: 3
initial_eligible_clients: 3
batch_size: 128
repeat: 1
seed: 0
server_warmup_epochs: 2
local_model_epochs: 15
weights_sending_frequency: 3
round_train_schedule_mode: auto
round_train_schedule_percentages: [] # manual mode: one percentage per round, all
rounds together must total 100
round_client_allocation_mode: auto
round_client_allocation_percentages: [] # manual mode: one row per round, one value per
client, each round totals 100

# Optimizer
server_learning_rate: 0.0005
learning_rate: 0.005 # scalar client LR
client_learning_rates: [0.005, 0.02, 0.1] # optional, one per client
momentum: 0.9

# Client churn
death_prob: 0.0
new_client_prob: 0.0

# Data distribution
server_data_percentage: 0.1
iid: false
imbalance_rate: 0.5
train_val_split: 0.9

# Method-specific fields
mu: 1.0 # fed_prox
fedprox_weighted: true # fed_prox: sample-weighted average (paper rule) vs. uniform
average
```

```
# GP fields (used by fed_gp)
individuals: 50
generations: 20
elitism_size: 1
mutation_rate: 0.4
crossover_rate: 0.6
gp_patience: 4
gp_initialization: genHalfAndHalf
mutation_type: mutUniform
mutation_subtree_maxsize: 2
selection_type: selTournament
crossover_type: cxOnePoint
min_tree_size: false
max_tree_size: 5
gp_transfer_learning: full_reuse
gp_fitness_metric: accuracy
available_primitives:
torch.add,torch.sub,torch.mul,torch_protected_div,torch_mean,torch_median,torch.abs,torch_protected_sqrt

# Custom model fields (optional)
custom_model_mode: default
custom_model_code: ''

# Custom dataset fields (optional; filled in automatically on upload
# when dataset_name is custom_csv, custom_image_npz, or custom_image_folder)
custom_dataset_ref: ''
custom_dataset_name: ''
```

Methods and Comparability

Supported methods:

- **FedAvg** (`fed_avg`): plain averaging of the client models.
- **FedAvgW** (`fed_avgw`): averaging weighted by how many samples each client trained on.
- **FedProx** (`fed_prox`): adds a regularization term that keeps clients close to the global model, which helps when client data differs a lot.
- **FedNova** (`fed_nova`): corrects for clients that train for different amounts before each aggregation.
- **FedGP** (`fed_gp`): an evolutionary aggregation strategy developed by our team.

Official method references:

- **FedAvg / FedAvgW**: [Communication-Efficient Learning of Deep Networks from Decentralized Data](#).
- **FedProx**: [Federated Optimization in Heterogeneous Networks](#).
- **FedNova**: [Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization](#).
- **FedGP**: [FedGP chapter reference](#) (method introduced by our team).

Single vs. multiple methods: select one method for a single run, or several methods to launch one independent run per method that share every other configuration field. In `config.yaml` this maps to the `methods` list. Pair it with `run_names`, one name per method in the same order, to label each run. This is the recommended way to compare methods on identical settings.

Parameter Reference

What each setting does. Expand a group to see its parameters.

Core parameters

Simple vs Advanced mode: Simple mode shows the core setup and handles round scheduling and client distribution automatically. Advanced mode adds churn controls, seed, per-round scheduling, per-client learning rates, custom models, repeats, and the full FedGP settings.

method: federated aggregation algorithm used for the run.

dataset_name: dataset used for training and evaluation.

num_clients: total number of clients in the run.

initial_eligible_clients: how many clients are active at the start, before churn. The lowest client IDs are used first.

batch_size: mini-batch size used in local training.

server_learning_rate: learning rate for the server warm-up before the rounds begin.

learning_rate: client learning rate. Must be positive.

client_learning_rates: one learning rate per client. Set it as Uniform, Manual, or Random in the UI.

momentum: momentum used by the local optimizer.

death_prob: chance, evaluated independently for each active client at the start of every round, that the client drops out.

new_client_prob: chance, evaluated independently for each inactive client at the start of every round, that the client rejoins. Dropout is not permanent: a client can drop and rejoin multiple times over a run, and the eligible set carries over from one round to the next rather than resetting.

seed: base seed for the whole run, covering data partitioning, training, and churn, so results are reproducible. Repeats use `seed + repeat_index` to stay independent.

server_warmup_epochs: number of server warm-up epochs before the rounds.

local_model_epochs: total epochs each client trains over the run.

weights_sending_frequency: how many local epochs a client trains before sending its weights to the server for aggregation.

Number of rounds: you do not set rounds directly. The number of aggregation rounds is `local_model_epochs ÷ weights_sending_frequency` (for example, 15 epochs at frequency 3 give 5 rounds), so the two values must divide evenly.

round_train_schedule_mode: how the training data is spread across rounds. `auto` spreads it evenly, `manual` lets you set the share per round, `no_split` reuses all the training data every round. Validation and test never change between rounds.

round_train_schedule_percentages: the per-round shares for `manual` mode, one positive value per round, totalling 100.

round_client_allocation_mode: how each round's data is divided among clients. `auto` keeps the same split every round; `manual` lets you set it per round. If a client drops out, its share is redistributed to the remaining active clients.

round_client_allocation_percentages: the per-client shares for `manual` mode, one value per client per round, each round totalling 100.

repeat: how many times to run the same setup (each repeat uses a different seed).

evaluation_split_mode: `train_val_test` evaluates on validation and test; `train_val` uses validation only.

Manual schedule percentages are a plan: the actual sample counts can shift slightly after rounding.

Data distribution parameters

server_data_percentage: share of the training data retained by the server (used for the server warm-up).

iid: toggles IID vs non-IID split of the training data across clients.

imbalance_rate: controls how unevenly training data is split across clients when `iid` is off, from more balanced near 0 to more skewed near 1. Must be strictly between 0 and 1; set `iid` to true for a fully balanced split.

train_val_split: train/validation ratio (e.g. `0.9`). Used only for FashionMNIST and CIFAR-10/100, where validation is carved out of the training data. MedMNIST and custom datasets already ship with their own splits, so the field is disabled for them.

Method-specific parameters

Most methods run on the shared settings above. Only FedProx adds parameters of its own; FedAvgW and FedNova adjust their aggregation automatically, with nothing extra to set.

mu (FedProx): strength of the proximal term that keeps each client close to the global model. Higher values pull clients back harder; `0` makes FedProx behave like plain averaging. Used only by FedProx.

fedprox_weighted (FedProx): checked averages clients weighted by how many samples each one trained on (the paper's rule, same as FedAvgW); unchecked uses a plain uniform average across clients. Used only by FedProx.

FedGP has its own group of settings, listed below.

GP parameters (fed_gp)

individuals: population size per generation.

generations: number of evolutionary iterations.

elitism_size: number of top individuals preserved unchanged.

mutation_rate: mutation probability.

crossover_rate: crossover probability.

gp_patience: no-improvement tolerance before early stop.

gp_initialization: initialization strategy for first population.

mutation_type: mutation operator choice.

mutation_subtree_maxsize: maximum height of the random subtree grafted in place of a mutated node when `mutation_type` is `mutUniform`. Lower values keep mutations small and limit tree bloat across generations. Integer from 1 to 64, default 2.

selection_type: parent selection strategy. `selRoulette` needs non-negative fitness, so it cannot be combined with `gp_fitness_metric: loss`.

crossover_type: crossover operator choice.

min_tree_size / max_tree_size: lower and upper complexity bounds.

gp_transfer_learning: controls reuse strategy across GP evolution phases. Valid values: `disabled`, `full_reuse`, `elite_reuse`, `elite_mutation_warm_start`, `hybrid`. The elite-based strategies carry over the top `elitism_size` individuals from the previous round, not just the single best, so the warm-started population keeps some diversity.

gp_fitness_metric: metric the evolution optimizes. `accuracy`, `precision`, `recall`, and `f1` are maximized (precision/recall/f1 are macro-averaged); `loss` is minimized. Default `accuracy`.

available_primitives: allowed primitive operation set. `torch_median` needs at least 3 clients to have an effect (the median of 2 values is their mean), so it is silently ignored below that; the setup form warns you when this applies.

Datasets and Upload Rules

FASTER supports built-in datasets and custom uploads via a unified management interface.

Built-in datasets are ready to use with no upload. They cover the MedMNIST family (`pathmnist`, `dermamnist`, `octmnist`, `bloodmnist`, `tissuemnist`, `pneumoniamnist`, `organamnist`, `organcmnist`, `organsmnist`) plus `fashionmnist`, `cifar10`, and `cifar100`. MedMNIST datasets already ship with their own train/val/test splits; FashionMNIST and CIFAR-10/100 carve the validation set out of the training data using `train_val_split`.

Custom datasets are anything you upload yourself, in one of three shapes: tabular (CSV/TSV), image arrays (NPZ), or an image folder archive (ZIP). The rules below apply to those uploads.

You upload one file and the platform detects its type. Accepted formats: `.csv`, `.tsv`, `.npz`, and `.zip`. The Datasets tab shows your own uploads;

The key rule: your upload must already define its own train, val, and test splits.

The platform never splits data for you. How you express the splits depends on the format:

- **CSV/TSV:** a header row, numeric feature columns, one target column, and a split column named `split`, `set`, or `partition` (see the example below).
- **NPZ:** explicit split arrays, either `train_*/val_*/test_*` or `x_train/x_val/x_test` with matching labels.
- **ZIP image folder:** `train/`, `val/`, and `test/` directories (optionally inside one wrapper folder), each containing one subfolder per class with the images directly inside. No extra nesting, traversal paths, or reserved/colliding filenames.

Uploads are validated immediately: empty splits, non-numeric features, invalid labels, or fewer than two classes are rejected with a clear message. Very large archives may be refused by the server (HTTP 413). Once an upload passes, the dataset is available in the Datasets tab and in the custom-dataset selector in Experiment Setup.

Tabular CSV example (click to expand)

```
feat_1,feat_2,feat_3,label,split
1.2,3.4,5.1,A,train
0.8,2.1,4.2,B,train
1.0,3.0,5.0,A,val
0.7,1.9,4.1,B,val
1.1,3.2,5.3,A,test
0.9,2.2,4.4,B,test
```

Accepted split aliases are: `train` / `training`, `val` / `valid` / `validation` / `dev`, and `test` / `testing`. The split column header can be named `split`, `set`, or `partition`.

The target column is detected by name (`label`, `target`, `class`, or `y`); if none is present, the last column is treated as the target. Every other column must be numeric. The dataset must contain at least two classes.

NPZ image example (click to expand)

```
# Required keys (option 1)
train_images, train_labels,
val_images, val_labels,
test_images, test_labels

# Or option 2
x_train, y_train,
x_val, y_val,
x_test, y_test
```

Flat keys like only `images` and `labels` are rejected because the platform does not split user datasets automatically.

ZIP image-folder example (click to expand)

```
dataset.zip
├─ root/
│   ├── train/
│   │   ├── class_a/
│   │   │   ├── img_001.png
│   │   │   └── img_002.png
│   │   └── class_b/
│   ├── val/
│   │   ├── class_a/
│   │   └── class_b/
│   └── test/
│       ├── class_a/
│       └── class_b/
```

Every split must contain the same class folders. The path to each image must be exactly `[wrapper/]split/class_name/image` with no extra folders in between. The archive is extracted and indexed once on upload, then reused on later runs.

Default Network Architectures

When `custom_model_mode` is `default`, the platform picks a built-in network for you based on the data: `TabularNet` for CSV/TSV datasets, and `Net` (a small CNN) for every image dataset (MedMNIST, FashionMNIST, CIFAR-10/100, NPZ, or ZIP folders).

What the default models expect:

- **Default image model:** image batches with integer labels. An adaptive pooling layer before the classifier normalizes the spatial size, so it handles both 28x28 (MedMNIST) and 32x32 (CIFAR) inputs. Images are converted to tensors and scaled, but not resized, so unusual image sizes may not fit.

- **Default tabular model:** numeric feature vectors with integer labels.

Simple mode always uses these defaults. If your data does not fit them (most often with ZIP image folders of unusual sizes), switch to Advanced mode and provide your own model. The exact code of both default networks is below for reference.

Default image architecture code (reference)

```
class Net(nn.Module):
    def __init__(self, in_channels, num_classes):
        super(Net, self).__init__()

        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels, 16, kernel_size=3),
            nn.BatchNorm2d(16),
            nn.ReLU())

        self.layer2 = nn.Sequential(
            nn.Conv2d(16, 16, kernel_size=3),
            nn.BatchNorm2d(16),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))

        self.layer3 = nn.Sequential(
            nn.Conv2d(16, 64, kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU())

        self.layer4 = nn.Sequential(
            nn.Conv2d(64, 64, kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU())

        self.layer5 = nn.Sequential(
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))

        # Normalize the conv output to a fixed 4x4 map so the classifier accepts
        # any input size (28x28 MedMNIST and 32x32 CIFAR alike).
        self.avgpool = nn.AdaptiveAvgPool2d((4, 4))

        self.fc = nn.Sequential(
            nn.Linear(64 * 4 * 4, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes))

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.layer5(x)
        x = self.avgpool(x)
```

```
x = x.view(x.size(0), -1)
x = self.fc(x)
return x
```

Default tabular architecture code (reference)

```
class TabularNet(nn.Module):
    def __init__(self, in_features, num_classes):
        super().__init__()
        hidden = max(32, min(256, in_features * 2))
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, num_classes),
        )

    def forward(self, x):
        if x.dim() > 2:
            x = x.view(x.size(0), -1)
        return self.net(x)
```

Custom Model Usage

When the default networks do not fit your data, switch to Advanced mode and set the model source to inline Python code. Your code is validated before launch and rejected with a clear message if it does not meet the rules below.

Required items and constraints

Mandatory symbol: define `MODEL_NAME` as a non-empty string.

Mandatory factory: define `build_model(n_channels, n_classes)`.

Factory contract: `build_model` must return a valid `torch.nn.Module` instance.

Valid syntax: the code must parse as Python.

Max size: 5 MB.

Blocked imports: `os`, `sys`, `subprocess`, `shutil`, `pathlib`, `socket`, `importlib`.

Blocked calls: `eval`, `exec`, `compile`, `__import__`.

Custom models work with every method, including FedGP. Run a short pilot before a long experiment, since a missing dependency only shows up when training actually starts.

Inline Python example (reference implementation)

```
import torch
import torch.nn as nn

MODEL_NAME = "CustomCNN16"

class ModelImpl(nn.Module):
    def __init__(self, n_channels, n_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(n_channels, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1)),
        )
        self.classifier = nn.Linear(32, n_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

def build_model(n_channels, n_classes):
    return ModelImpl(n_channels, n_classes)
```

Your model code is saved with the run, so its results stay reproducible from History.

Scenario Management

A scenario is a named, reusable preset of a full run setup. Unlike Save Defaults (which prefills the form automatically), a scenario is loaded on demand, so you can keep several side by side.

- **Load:** fill the form from a saved scenario. Simple or Advanced mode is selected automatically to match it.
- **Save:** store the current setup as a named scenario.
- **Import YAML:** fill the form from an external YAML file. An optional `ui_mode: simple` or `ui_mode: advanced` line is respected if present.
- **Review before launch:** the summary shows only the settings that apply to your chosen methods (for example, GP fields are hidden unless you use FedGP).

Live Observability

Live Monitor shows progress, metrics, and logs while a run is training. The metric cards report accuracy, loss, precision, recall, F1, and ROC AUC. They show the latest value at each point, not a final summary.

Validation metrics are always shown. Test metrics appear only for `train_val_test` runs. ROC AUC can read `N/A` when the data does not have enough class structure to compute it.

How each metric is computed

All metrics are reported on a 0 to 1 scale, computed on the evaluated split (validation, and test when enabled).

Accuracy: fraction of correctly classified samples.

Loss: cross-entropy, averaged over the samples in the split.

Precision, recall, F1: macro-averaged, meaning each class counts equally regardless of how many samples it has. This is the right choice for imbalanced data, but it penalizes rare classes that the model handles poorly.

ROC AUC: for two classes, the standard binary score; for more than two, the one-vs-rest average. It needs at least two classes present in the evaluated data, otherwise it shows `N/A`.

History and Comparison

History is where you inspect finished runs, export them, and compare them. A run appears here as soon as it is submitted, before it has produced any results.

- Select **one** run to open its detail view and download its export.
- Select **2 to 5** runs and use Compare to chart them together.

A single run's detail view shows its final validation and test values. A comparison highlights the best value reached for each metric, and only includes test metrics when every selected run has them.

Repeated runs (repeat > 1): the charts and final metric cards show the mean across repeats, and the detail view reports how many repeats ran. The summary table lists each repeat plus a Mean $\pm$ Std row, and the export ZIP contains the full per-repeat data (metrics, logs, and saved models), not just the average.

What the export ZIP contains

Exporting a run downloads its full results folder, so you can analyze it outside the platform and keep a complete record:

- `config.yaml` : the exact configuration the run used.
- `metadata.json` : run metadata, including the resolved settings.
- `metrics.json` : every metric for every round, for each evaluated split.
- `global_model_<round>.pth` : the aggregated model saved after each round.
- `train.log` : the full run log.
- class-distribution plots for the training data.

For repeated runs, each repeat has its own `repeat_<n>/` folder with the same files, plus a top-level `summary_metrics.json` with the aggregate across repeats.

System Status

System Status shows current hardware use and the run queue at a glance:

- **Hardware:** live CPU, RAM, and GPU usage.
- **Jobs:** active and queued runs, with the actions you can take on them.
- **Per-run usage:** average hardware usage for a finished run, in its detail view.

Reproducibility

Each run is driven by its `seed`, which fixes the random parts of the experiment: how data is partitioned across clients, the training itself, and which clients drop out or rejoin each round. Two runs with the same configuration and the same seed follow the same random path. Repeats reuse the base seed plus the repeat index, so the repeats of one run are independent of each other but reproducible as a set.

The exported `config.yaml` and seed let anyone rebuild the exact setup of a run.

Troubleshooting

- **Run does not start:** check the validation messages on the Setup form for the field that needs fixing.
- **No live updates:** confirm the run is still active in System Status, then reopen Live Monitor.
- **No comparison charts:** select at least 2 runs.
- **Export unavailable:** make sure the run has finished.

When reporting an issue, include the Run ID, method, dataset, and evaluation mode.